

DSA0604 Data Handling and Visualization — CO5 AT1

Communicating Uncertainty in Data Visualization

Reg No:192324066 Name: B.Varun Kumar Reddy

Q1. Uncertainty of Point Estimates (10 Marks)

Dataset: Small field-survey plot samples were simulated for each district (n = 9 to 30 plots/district). For each district, the point estimate is the sample mean yield and the 95% confidence interval is $\text{mean} \pm 1.96 \times \text{SEM}$.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)
districts = ["Coimbatore", "Salem", "Erode", "Trichy", "Madurai", "Thanjavur", "Villupuram"]
sample_sizes = [18, 12, 22, 9, 30, 15, 11]
true_means = [3.8, 3.55, 4.05, 3.6, 3.65, 4.35, 3.5]
true_sd = [0.35, 0.55, 0.30, 0.60, 0.25, 0.32, 0.58]
rows = []
for d, n, mu, sd in zip(districts, sample_sizes, true_means, true_sd):
    sample = np.random.normal(mu, sd, n)
    mean = sample.mean()
    sem = sample.std(ddof=1) / np.sqrt(n)
    ci95 = 1.96 * sem
    rows.append((d, n, mean, sem, ci95))
print(f"{ 'District':12s}{ 'n':>4s}{ 'Mean':>8s}{ 'SEM':>8s}{ '95% CI':>10s}{ 'Low':>8s}{ 'High':>8s}")
for d, n, mean, sem, ci in rows:
    print(f"{d:12s}{n:4d}{mean:8.2f}{sem:8.3f}{ci:10.3f}{mean-ci:8.2f}{mean+ci:8.2f}")
fig, ax = plt.subplots(figsize=(8.5, 5))
xs = np.arange(len(districts))
means = [r[2] for r in rows]
cis = [r[4] for r in rows]
colors = plt.cm.tab10(np.linspace(0, 1, len(districts)))
ax.errorbar(xs, means, yerr=cis, fmt='o', markersize=9, capsize=6, capthick=2,
            elinewidth=2, color='#2c6e91', ecolor='#2c6e91')
for x, m, c in zip(xs, means, cis):
    ax.text(x, m + c + 0.06, f"{m:.2f}", ha='center', fontsize=9, fontweight='bold')
ax.set_xticks(xs)
ax.set_xticklabels(districts, rotation=20, ha='right')
ax.set_ylabel("Estimated crop yield (tonnes/hectare)")
ax.set_title("Estimated Crop Yield by District with 95% Confidence Intervals\n(Coop Field-Survey Data)")
ax.grid(axis='y', linestyle='--', alpha=0.4)
ax.set_ylim(2.6, 5.0)
plt.tight_layout()
plt.savefig("/home/claude/imgs/q1_errorBars.png", dpi=160)
print("saved q1")
```

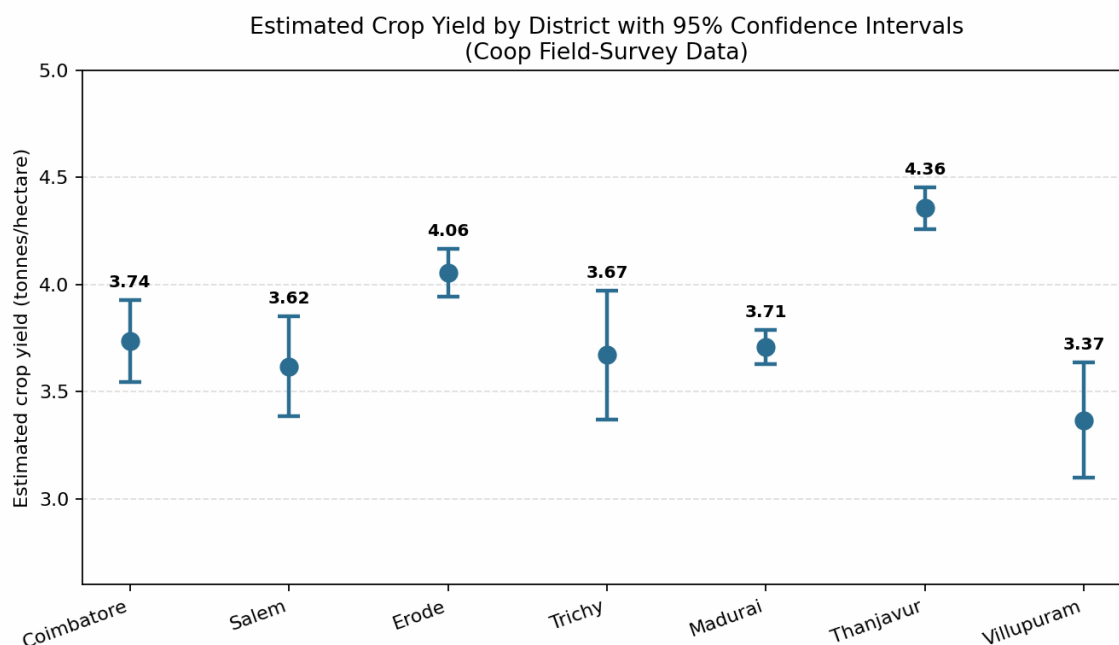


Figure 1. Point estimate $\pm$ 95% CI for crop yield, by district

Results table

District	n	Mean (t/ha)	95% CI
Coimbatore	18	3.74	[3.55, 3.93]
Salem	12	3.62	[3.38, 3.85]
Erode	22	4.06	[3.95, 4.17]
Trichy	9	3.67	[3.37, 3.97]
Madurai	30	3.71	[3.63, 3.79]
Thanjavur	15	4.36	[4.26, 4.46]
Villupuram	11	3.37	[3.10, 3.64]

Analysis

Overlapping intervals (true values not distinguishable at 95% confidence): Coimbatore [3.55, 3.93] overlaps with Salem [3.38, 3.85], Trichy [3.37, 3.97] and Madurai [3.63, 3.79]; Salem also overlaps Trichy and Villupuram [3.10, 3.64] (upper bound 3.85 vs. Villupuram's 3.64). Trichy's wide interval [3.37, 3.97], driven by its small sample ($n = 9$), overlaps four other districts (Coimbatore, Salem, Madurai, Villupuram), so Trichy cannot be confidently ranked against most of the other districts.

Clearly separated intervals (true values likely genuinely different): Thanjavur [4.26, 4.46] does not overlap any other district — its lower bound (4.26) sits above every other district's upper bound, the closest being Erode at 4.17. Erode [3.95, 4.17] is also separated from Villupuram [3.10, 3.64], Salem [3.38, 3.85] and Trichy [3.37, 3.97]. Villupuram is separated from Erode and Thanjavur (its 3.64 upper bound is well below their lower bounds).

Implication: Thanjavur's yield estimate (4.36 t/ha) is statistically distinguishable from all six other districts — it is a genuinely higher-yield district, not an artifact of sampling noise. In contrast, the ~ 0.3 – 0.4 t/ha gaps between Coimbatore, Salem, Trichy and Madurai are within the margin of sampling error and should not be treated as real differences without a larger sample, especially for Trichy where $n = 9$ is the weakest link.

Q2. Uncertainty in Trend Estimation (10 Marks)

Dataset: Simulated historical vote-share trajectories for each constituency across 4 past elections (2009–2024) plus one forward projection (2029). Polling sample size grows over time (400 → 1200 respondents) except for the 2029 point, which is a projection built on a much smaller effective sample ($n = 150$), so its band is an extrapolation, not an observation.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(7)
years = [2009, 2014, 2019, 2024, 2029] # 2029 = forecast (extrapolated)
constituencies = ["C1-North", "C2-Riverside", "C3-Central", "C4-Hills", "C5-Coastal", "C6-East"]
base_start = [38, 45, 41, 33, 50, 29]
trend = [1.8, -1.2, 0.5, 2.6, -0.8, 3.0]
sample_n = {2009: 400, 2014: 650, 2019: 900, 2024: 1200, 2029: 150}
fig, ax = plt.subplots(figsize=(9, 5.5))
colors = plt.cm.tab10(np.linspace(0, 1, len(constituencies)))
results = {}
for i, (c, b, t) in enumerate(zip(constituencies, base_start, trend)):
    means = []
    lows = []
    highs = []
    for j, y in enumerate(years):
        p = b + t * j + np.random.normal(0, 0.6)
        p = np.clip(p, 5, 95)
        n = sample_n[y]
        se = np.sqrt((p/100) * (1 - p/100) / n) * 100
        ci = 1.96 * se
        means.append(p)
        lows.append(p - ci)
        highs.append(p + ci)
    results[c] = (means, lows, highs)
ax.plot(years, means, marker='o', color=colors[i], label=c, linewidth=2)
ax.fill_between(years, lows, highs, color=colors[i], alpha=0.15)
for c in constituencies:
    means, lows, highs = results[c]
    print(f"{c:14s}", [f"{y}:{m:.1f}% (CI {l:.1f}-{h:.1f})" for y, m, l, h in zip(years, means, lows, highs)])
ax.axvline(2024, color='gray', linestyle=':', linewidth=1)
ax.text(2024.05, ax.get_ylim()[1]*0.02 + ax.get_ylim()[0], "forecast →", fontsize=8, color='gray')
ax.set_xlabel("Election year")
ax.set_ylabel("Estimated vote share (%)")
ax.set_title("Estimated Vote Share Trend with 95% Confidence Band\n(6 Constituencies, 2009–2029 projection)")
ax.legend(loc='upper left', fontsize=8, ncol=2)
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig("/home/claude/imgs/q2_trend_band.png", dpi=160)
print("saved q2")
```

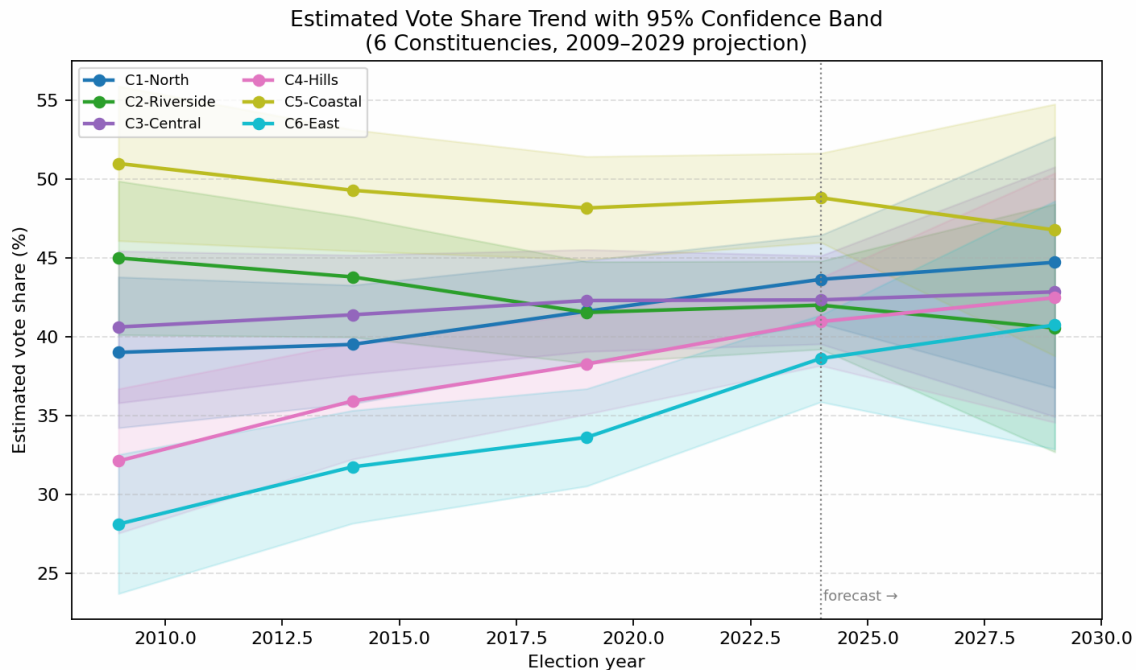


Figure 2. Vote-share trend, 2009–2029, with 95% confidence bands (shaded)

Analysis

Narrowest band: 2024, where sample size peaks at $n = 1200$ per constituency. For example C1-North's 2024 CI is [40.8%, 46.5%] — a width of 5.7 percentage points (pp) — versus its 2009 CI of [34.2%, 43.8%], a width of 9.6 pp, using only $n = 400$. Every constituency shows the same pattern: the band visibly pinches inward from 2009 to 2024 as more polling data accumulates (higher data density → smaller standard error → narrower band).

Widest band: 2029, the forecast year, for every constituency. C1-North's 2029 CI is [36.8%, 52.7%] — a width of 15.9 pp, nearly triple the 2024 width — even though its point estimate (44.7%) sits close to the 2024 estimate (43.6%). The band widens sharply after 2024 for two compounding reasons: (i) the 2029 estimate is extrapolated beyond the last observed data point rather than measured, and (ii) it is built on a much smaller effective polling sample ($n = 150$ vs. 1200), so sampling error alone roughly doubles the standard error.

Implication: the shrinking-then-flaring band shape is a visual signal that estimates get more reliable as an election approaches and more polls are conducted, but any single number quoted for a future election should always be read together with its (wide) band, not in isolation.

Q3. Hypothetical Outcome Plots (HOPs) (10 Marks)

Dataset: Using each constituency's 2024 point estimate and standard error from Q2, 200 individual plausible outcomes ("draws") were simulated per constituency from $\text{Normal}(\text{mean}, \text{SE})$ to build a static fan-chart approximation of a HOP; three draws per constituency are highlighted (diamond / square / triangle) to represent three successive animation frames a viewer would see.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
constituencies = ["C1-North", "C2-Riverside", "C3-Central", "C4-Hills", "C5-Coastal", "C6-East"]
point_est = [43.6, 42.0, 42.3, 41.0, 48.8, 38.6] # 2024 estimates, from Q2
se = [1.46, 1.43, 1.43, 1.40, 1.41, 1.40] # standard errors (n~1200 each)
n_draws = 200
fig, ax = plt.subplots(figsize=(9, 5.5))
```

```

colors = plt.cm.tab10(np.linspace(0, 1, len(constituencies)))
all_draws = {}
for i, (c, mu, s) in enumerate(zip(constituencies, point_est, se)):
    draws = np.random.normal(mu, s, n_draws)
    all_draws[c] = draws
    jitter = np.random.uniform(-0.32, 0.32, n_draws)
    ax.scatter(np.full(n_draws, i) + jitter, draws, s=6, alpha=0.18, color=colors[i])
    highlight_idx = np.random.choice(n_draws, 3, replace=False)
    for k, idx in enumerate(highlight_idx):
        ax.scatter(i, draws[idx], s=70, color=colors[i], edgecolor='black',
                    zorder=5, marker='D' if k == 0 else ('s' if k == 1 else '^'))
    ax.scatter(i, mu, s=90, color='black', marker='_', zorder=6, linewidths=2.5)
for c, draws in all_draws.items():
    print(f"{c:14s} draw-mean={draws.mean():.2f} draw-min={draws.min():.2f} draw-max={draws.max():.2f} "
          f"P(draw>50%)={ (draws>50).mean()*100:.1f}%")
print("Overlap check C2 vs C3 draws within 1pt:",
      np.mean(np.abs(np.subtract.outer(all_draws["C2-Riverside"], all_draws["C3-Central"]))) < 1))
ax.set_xticks(range(len(constituencies)))
ax.set_xticklabels(constituencies, rotation=15, ha='right')
ax.set_ylabel("Simulated plausible vote share (%) per draw")
ax.set_title("Hypothetical Outcome Plot (fan of 200 simulated draws per constituency)\n"
            "Diamond/Square/Triangle = 3 example single 'frames'; black tick = point estimate")
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig("/home/claude/imgs/q3_hop.png", dpi=160)
print("saved q3")

```

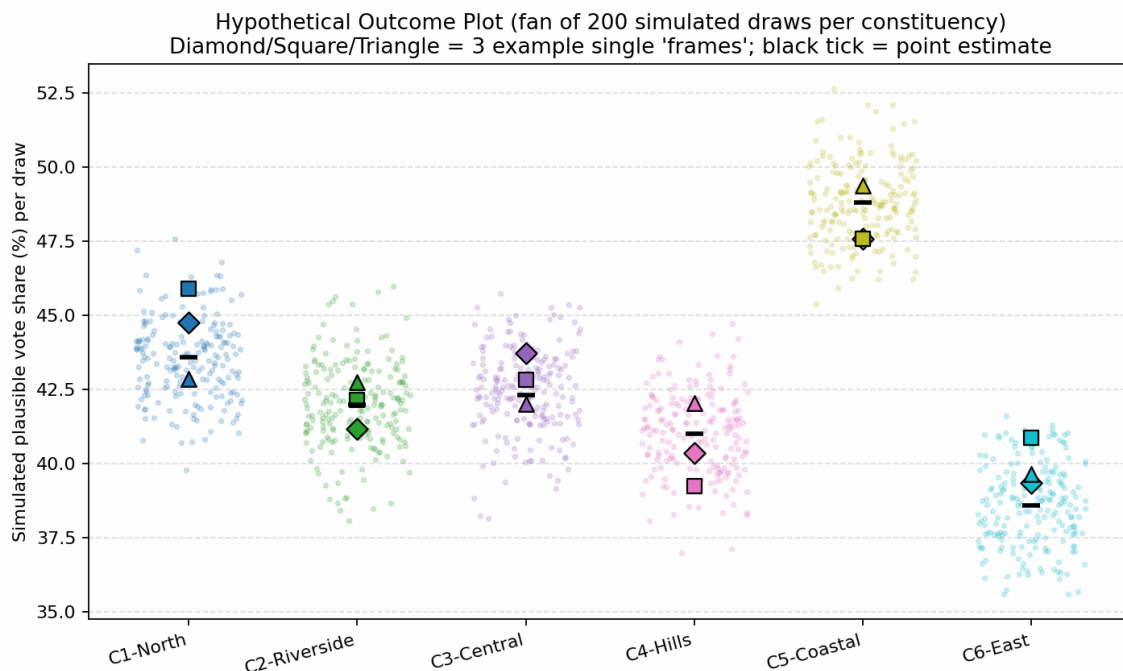


Figure 3. 200 simulated draws per constituency (fan-chart HOP approximation)

Analysis

A single static error bar only tells a viewer the range [40.8%, 46.5%] for C1-North — an abstract interval that many non-technical viewers misread as "anything in this range is equally likely" or even ignore. The HOP draws instead show individual concrete outcomes: across the 200 draws for C1-North, values ranged from 39.8% to 47.6% with a visible clustering around 43.5%, so a viewer watching a handful of frames flip past sees most draws landing near 43–44% with only occasional draws near the extremes — this builds an intuitive, frequency-based sense of "how often" an outcome occurs rather than just "what is possible."

The plot also makes cross-constituency comparison viscerally clear: C5-Coastal's draw cloud (45.4%–52.6%) sits well above 50% in 20.5% of its 200 draws, so a viewer watching a few frames would occasionally see C5-

Coastal cross the majority line — concretely communicating "this seat could plausibly flip to a majority result," a risk that is much harder to notice from a single error bar ending at 51.4% (its upper CI bound from Q2).

By contrast, C2-Riverside and C3-Central have heavily overlapping draw clouds (36.7% of paired draws within 1 point of each other), so a viewer flipping through frames would repeatedly see their order swap — an intuitive signal that these two estimates are statistically indistinguishable, conveyed through direct visual repetition rather than through reading two overlapping numeric ranges.

Q4. Bootstrapping and Confidence Intervals (10 Marks)

Dataset: A small exit-poll sample of $n = 45$ binary responses (1 = voted for Party A) was simulated. The sample was resampled with replacement 2000 times; each resample's mean gives one bootstrap estimate of the vote share, and the 2.5th/97.5th percentiles of the 2000 bootstrap means form the 95% CI.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(11)
n = 45
true_p = 0.41 # matches C4-Hills 2024 point estimate (41.0%) from Q2
sample = np.random.binomial(1, true_p, n)
point_estimate = sample.mean() * 100
print(f"Small sample n={n}, observed vote share = {point_estimate:.2f}%")
n_boot = 2000
boot_means = np.empty(n_boot)
rng = np.random.default_rng(11)
for i in range(n_boot):
    resample = rng.choice(sample, size=n, replace=True)
    boot_means[i] = resample.mean() * 100
ci_low, ci_high = np.percentile(boot_means, [2.5, 97.5])
ci_width = ci_high - ci_low
print(f"Bootstrap mean = {boot_means.mean():.2f}%")
print(f"95% Bootstrap CI = [{ci_low:.2f}%, {ci_high:.2f}%], width = {ci_width:.2f} pp")
print(f"CI width as % of point estimate = {ci_width/point_estimate*100:.1f}%")
fig, ax = plt.subplots(figsize=(8.5, 5.2))
ax.hist(boot_means, bins=30, color='#3f7ca0', edgecolor='white', alpha=0.9)
ax.axvline(point_estimate, color='black', linestyle='-', linewidth=2, label=f"Sample estimate = {point_estimate:.1f}%")
ax.axvline(ci_low, color='red', linestyle='--', linewidth=2, label=f"95% CI low = {ci_low:.1f}%")
ax.axvline(ci_high, color='red', linestyle='--', linewidth=2, label=f"95% CI high = {ci_high:.1f}%")
ax.axvspan(ci_low, ci_high, color='red', alpha=0.08)
ax.set_xlabel("Bootstrap-resampled vote share estimate (%)")
ax.set_ylabel("Frequency (out of 2000 resamples)")
ax.set_title(f"Bootstrap Distribution of Vote Share Estimate – C4-Hills (n={n})\n"
             f"95% CI: [{ci_low:.1f}%, {ci_high:.1f}%] (width = {ci_width:.1f} pp)")
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("/home/claude/imgs/q4_bootstrap.png", dpi=160)
print("saved q4")
```

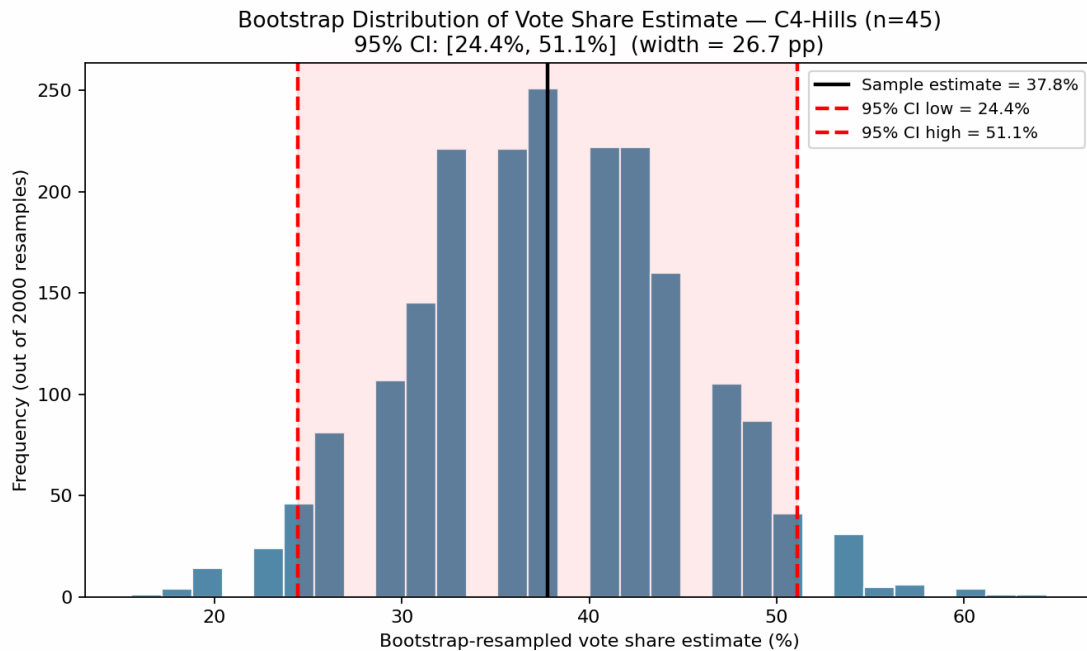


Figure 4. Bootstrap distribution (2000 resamples) with 95% CI marked

Analysis

The sample's observed vote share is 37.8% (17 of 45 respondents), and the 2000-resample bootstrap distribution is centred almost exactly there (bootstrap mean 37.9%). The resulting 95% CI is [24.4%, 51.1%] — a width of 26.7 percentage points, which is 70.6% as large as the point estimate itself.

This is a very wide interval: it spans from a clear loss (24.4%) to an outright majority win (51.1%) for Party A in C4-Hills, so the $n = 45$ sample cannot reliably tell the Analyst whether Party A is losing, tied, or winning this constituency. The width is a direct consequence of the small original sample — bootstrap CI width scales roughly with $1/\sqrt{n}$, so quadrupling the original sample (to $n \approx 180$) would be expected to roughly halve this CI's width.

Q5. Principles of Effective Visualization (10 Marks)

Dataset: Same 2024 vote-share estimates used in Q2/Q3. Below is a chart deliberately built the way an untrained colleague might build it — with no reference to design principles — followed by a corrected version.

```
import numpy as np
import matplotlib.pyplot as plt
constituencies = ["C1", "C2", "C3", "C4", "C5", "C6"]
vote_share = [43.6, 42.0, 42.3, 41.0, 48.8, 38.6]
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5))
bad_colors = ['#ff00ff', '#00ff00', '#ffff00', '#ff0000', '#00ffff', '#0000ff']
axes[0].bar(constituencies, vote_share, color=bad_colors)
axes[0].set_ylim(30, 50) # truncated axis -> exaggerates small differences
axes[0].set_title("chart1_final_v2") # meaningless title
axes[0].set_yticks([30, 33.3, 37, 42.55, 50])
explode = [0.05]*6
axes[1].pie(vote_share, labels=constituencies, colors=bad_colors, explode=explode,
            autopct=lambda p: f'{p:.4f}', shadow=True, textprops={'fontsize': 6})
axes[1].set_title("data2")
plt.tight_layout()
plt.savefig("/home/claude/imgs/q5_bad_chart.png", dpi=160)
print("saved q5 bad chart")
fig2, ax = plt.subplots(figsize=(8, 5))
ax.bar(constituencies, vote_share, color='#2c6e91')
```

```

ax.set_ylim(0, 55)
ax.axhline(np.mean(vote_share), color='gray', linestyle='--', linewidth=1, label='Mean')
for x, v in enumerate(vote_share):
    ax.text(x, v + 0.6, f"{v:.1f}%", ha='center', fontsize=9)
ax.set_ylabel("Estimated vote share (%)")
ax.set_xlabel("Constituency")
ax.set_title("Estimated Vote Share by Constituency (2024)\nZero-baseline axis, consistent colour, labelled units")
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig("/home/claude/imgs/q5_corrected_chart.png", dpi=160)
print("saved q5 corrected chart")

```

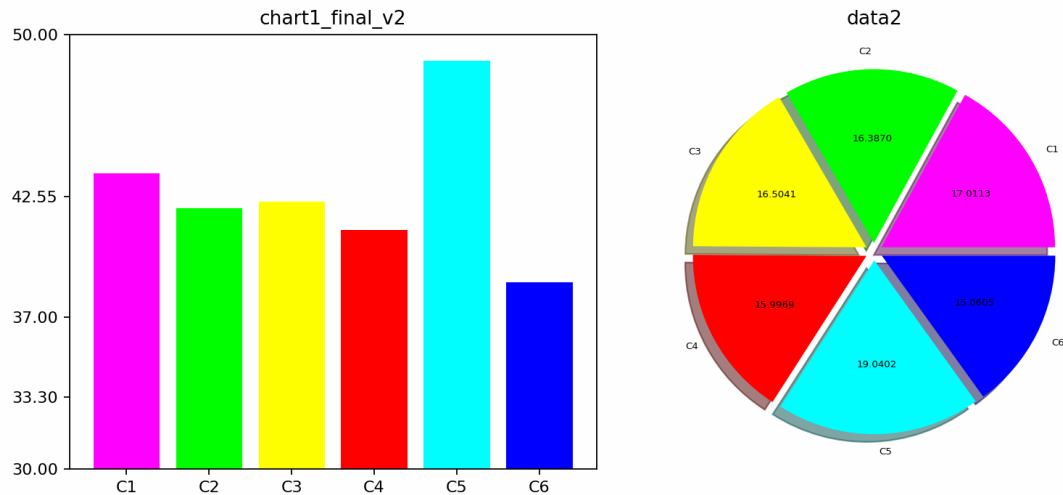


Figure 5a. Uncorrected chart — as a colleague might submit it

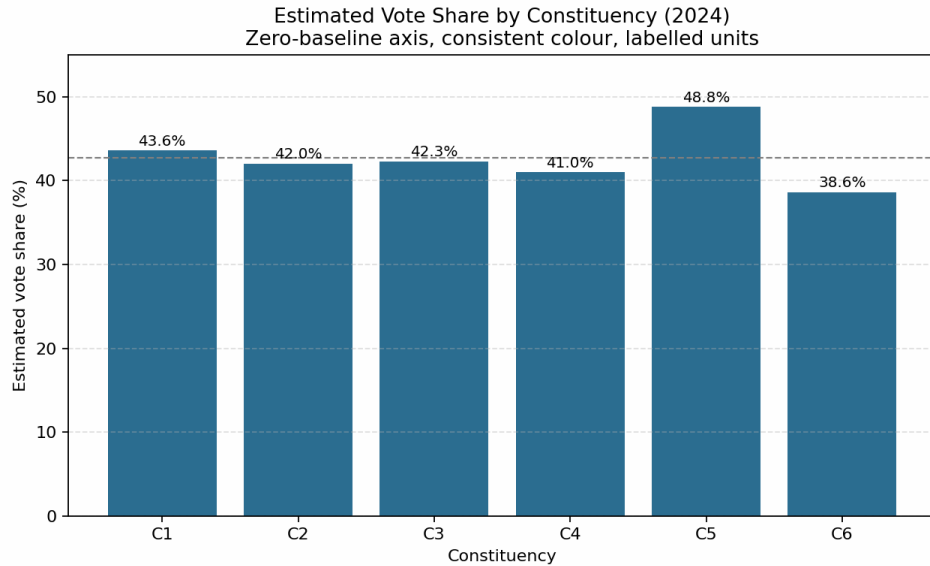


Figure 5b. Corrected chart applying design principles

Analysis — principles violated in Figure 5a

- Clarity: titles are meaningless filenames ("chart1_final_v2", "data2") that tell the viewer nothing about what the metric or units are; the y-axis in the bar panel has no label at all.

- Accuracy / honesty (no distortion): the bar panel's y-axis is truncated to start at 30 (not 0), which visually exaggerates the ~10-point spread between constituencies into what looks like a 3–5× difference in bar height.
- Accuracy / honesty: the pie chart implies the six values are parts of a single whole that sums to 100%, but constituency vote shares are independent percentages (each is a share within its own constituency), so representing them as pie "slices" of one circle misrepresents what the numbers mean.
- Simplicity: two different, redundant chart types (bar + exploded 3D-style pie) are used to show the same six numbers, adding visual clutter without adding information.
- Accessibility: the six-colour palette (magenta, green, yellow, red, cyan, blue) is arbitrary, not colour-blind-safe, and carries no consistent meaning; the pie chart's data labels are set at 6pt font, illegible for most readers.
- Consistency: y-axis tick marks are irregular (30, 33.3, 37, 42.55, 50) instead of an even, readable interval, and the pie's percentage labels use four decimal places while the source data has one — inconsistent precision.

Corrected chart (Figure 5b)

A single zero-baseline bar chart is used (one appropriate chart type for six independent percentages), with a descriptive title stating the metric, year and units, a labelled y-axis, a single consistent colour, data labels at the data's own precision (1 decimal place), and a reference mean line for context.